



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش پروژه درس معماری کامپیوتر
نیم سال دوم ۱۴۰۴-۰۵

واحد پیش واکشی حافظه

نگارش

آتنا رئیسی (۴۰۳۱۰۵۹۵۲)

شادی قابل (۴۰۳۱۷۱۱۱۳)

پرینا دباغ (۴۰۳۱۱۰۶۴۷)

استاد

دکتر سربازی آزاد

مرداد ۱۴۰۵

فهرست مطالب

۱	مقدمه	۱
۱	۱-۱ بیان مسئله و اهمیت موضوع	۱
۲	۲-۱ اهداف پروژه	۲
۲	۳-۱ ساختار گزارش	۲
۳	۲ مفاهیم پایه و معرفی شبیه ساز	۳
۳	۱-۲ حافظه نهان و سلسله مراتب حافظه	۳
۴	۲-۲ پیش واکشی حافظه	۴
۴	۳-۲ الگوریتم پیش واکشی گام دار	۴
۵	۴-۲ ترجمه آدرس و TLB	۵
۵	۵-۲ ساختار MSHR	۵
۵	۶-۲ معرفی شبیه ساز Akita	۵
۶	۷-۲ ساختار حافظه نهان در Akita	۶
۷	۳ طراحی و پیاده سازی واحد پیش واکشی گام دار	۷
۷	۱-۳ بررسی مسیر پردازش درخواست و محل اتصال پیش واکشی	۷
۸	۲-۳ معماری کلی واحد پیش واکشی	۸
۹	۳-۳ رابط و پیکربندی پیش واکشی کننده	۹
۱۰	۴-۳ پیاده سازی الگوریتم پیش واکشی گام دار	۱۰

۱۰	۳-۴-۱ وضعیت هر جریان و تاریخچه دسترسی ها
۱۱	۳-۴-۲ هم تراز کردن آدرس ها
۱۱	۳-۴-۳ محاسبه گام و به روز رسانی اطمینان
۱۲	۳-۴-۴ تولید آدرس های پیش واکشی
۱۳	۳-۵ اتصال الگوریتم به حافظه نهان سطح دوم
۱۴	۳-۶ ایجاد و پردازش تراکنش های پیش واکشی
۱۵	۳-۷ ثبت پیش واکشی های مفید، آلودگی و شمارنده های آماری
۱۶	۳-۸ آزمون های صحت پیاده سازی
۱۷	۳-۹ دسترسی به کد منبع

۴ ارزیابی و تحلیل نتایج

۱۸	۴-۱ روش انجام آزمایش ها
۱۹	۴-۲ برنامه محک و تنظیمات اجرا
۲۰	۴-۳ معیارهای ارزیابی
۲۱	۴-۴ مقایسه زمان اجرا
۲۲	۴-۵ مقایسه نرخ اصابت و نقصان حافظه نهان
۲۳	۴-۶ مقایسه نرخ اصابت و نقصان MSHR
۲۳	۴-۷ بررسی پوشش پیش واکشی
۲۴	۴-۸ بررسی میزان استفاده از داده های پیش واکشی شده
۲۵	۴-۹ بررسی آلودگی حافظه نهان
۲۵	۴-۱۰ تحلیل کلی نتایج
۲۷	۴-۱۱ اثبات درستی تشخیص گام

۵ جمع بندی و پیشنهادها

۲۹	۵-۱ جمع بندی پروژه
----	--------------------

۲۹	 محدودیت‌های پروژه	۲-۵
۳۰	 پیشنهادها برای توسعه آینده	۳-۵

فصل ۱

مقدمه

در معماری رایانه، سرعت پردازنده‌ها بسیار بیشتر از سرعت حافظه اصلی است. این اختلاف سرعت باعث می‌شود که دسترسی به داده‌های موجود در حافظه، یکی از عوامل مهم کاهش کارایی سامانه باشد. هنگامی که داده مورد نیاز پردازنده در حافظه نهان وجود نداشته باشد، پردازنده باید تا زمان دریافت آن از سطح پایین‌تر حافظه منتظر بماند. به این وضعیت، نقصان یا عدم اصابت حافظه نهان گفته می‌شود.

یکی از روش‌های کاهش این زمان انتظار، استفاده از پیش‌واکشی حافظه است. در این روش، درخواست‌های حافظه بررسی می‌شوند و داده‌هایی که احتمال دارد در آینده مورد استفاده قرار گیرند، پیش از درخواست پردازنده به حافظه نهان منتقل می‌شوند. در این پروژه، یک واحد پیش‌واکشی برای حافظه نهان سطح دوم در محیط شبیه‌سازی Akita طراحی و پیاده‌سازی شده است.

۱-۱ بیان مسئله و اهمیت موضوع

هنگام رخ دادن نقصان در حافظه نهان، اجرای برنامه تا زمان دریافت داده مورد نیاز با تأخیر مواجه می‌شود. در برنامه‌هایی که تعداد زیادی دسترسی به حافظه دارند، این تأخیرها می‌توانند زمان اجرای برنامه را به میزان قابل توجهی افزایش دهند.

واحد پیش‌واکشی با بررسی الگوی درخواست‌های خواندن حافظه، آدرس درخواست‌های آینده را پیش‌بینی می‌کند. در صورت تشخیص یک الگوی منظم، داده‌های احتمالی پیش از نیاز پردازنده به حافظه نهان سطح دوم منتقل می‌شوند. در نتیجه، ممکن است درخواست بعدی پردازنده به جای نقصان، با اصابت در حافظه نهان پاسخ داده شود.

با این حال، پیش‌واکشی همیشه باعث بهبود عملکرد نمی‌شود. پیش‌بینی اشتباه می‌تواند داده‌های مفید را از حافظه نهان خارج کند و باعث آلودگی حافظه نهان شود. همچنین، درخواست‌های اضافی پیش‌واکشی پهنای باند حافظه را مصرف می‌کنند. بنابراین، واحد پیش‌واکشی باید به گونه‌ای طراحی شود که میان کاهش تأخیر حافظه و سربار ایجادشده تعادل برقرار کند.

۲-۱ اهداف پروژه

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک واحد پیش‌واکشی گام‌دار یا Stride Prefetcher در حافظه نهان سطح دوم شبیه‌ساز Akita است. این واحد باید درخواست‌های ورودی حافظه را بررسی کرده و الگوهای دسترسی با فاصله ثابت را تشخیص دهد.

در الگوریتم پیاده‌سازی‌شده، فاصله میان آدرس‌های متوالی محاسبه می‌شود و پس از تکرار یک فاصله مشخص، درخواست‌های پیش‌واکشی تولید می‌شوند. همچنین، امکان تنظیم پارامترهایی مانند تعداد درخواست‌های پیش‌واکشی، میزان اطمینان لازم برای تشخیص الگو و اندازه تاریخچه دسترسی‌ها فراهم شده است.

از دیگر اهداف پروژه می‌توان به امکان فعال یا غیرفعال کردن واحد پیش‌واکشی، جلوگیری از تولید درخواست‌های نامعتبر، پشتیبانی از گام‌های مثبت و منفی و ارزیابی تأثیر پیش‌واکشی بر زمان اجرا و عملکرد زیرسامانه حافظه اشاره کرد.

۳-۱ ساختار گزارش

در فصل دوم، مفاهیم پایه Akita معرفی می‌شوند.

در فصل سوم، طراحی و پیاده‌سازی واحد پیش‌واکشی، نحوه اتصال آن به حافظه نهان سطح دوم، پارامترهای الگوریتم و تغییرات انجام‌شده در شبیه‌ساز توضیح داده می‌شوند.

در فصل چهارم، روش اجرای آزمایش‌ها، برنامه‌های محک، آزمون‌های انجام‌شده و معیارهای ارزیابی معرفی می‌شوند. سپس نتایج مربوط به زمان اجرا، نرخ نقصان، نرخ اصابت MSHR، پوشش، میزان استفاده از پیش‌واکشی‌ها و آلودگی حافظه نهان بررسی خواهند شد.

در فصل پایانی نیز نتایج کلی پروژه جمع‌بندی شده و محدودیت‌های پیاده‌سازی و پیشنهادهایی برای بهبود واحد پیش‌واکشی در آینده ارائه می‌شوند.

فصل ۲

مفاهیم پایه و معرفی شبیه‌ساز

برای درک نحوه طراحی واحد پیش‌واکشی، لازم است ابتدا با عملکرد حافظه نهان، پیش‌واکشی حافظه و ساختار شبیه‌ساز مورد استفاده در پروژه آشنا شویم. در این فصل، مفاهیم مورد نیاز به صورت خلاصه معرفی می‌شوند.

۱-۲ حافظه نهان و سلسله‌مراتب حافظه

سرعت پردازنده بسیار بیشتر از سرعت حافظه اصلی است. بنابراین، مراجعه مستقیم پردازنده به حافظه اصلی برای هر درخواست می‌تواند زمان اجرای برنامه را افزایش دهد. برای کاهش این اختلاف سرعت، سامانه حافظه به صورت سلسله‌مراتبی طراحی می‌شود و حافظه‌های سریع‌تر و کوچک‌تر در نزدیکی پردازنده قرار می‌گیرند.

حافظه نهان یا Cache یکی از سطوح این سلسله‌مراتب است و داده‌هایی را نگهداری می‌کند که اخیراً استفاده شده‌اند یا احتمال استفاده مجدد از آن‌ها زیاد است. داده‌ها در حافظه نهان به صورت واحدهایی با اندازه ثابت به نام بلوک یا خط حافظه نهان ذخیره می‌شوند.

اگر داده مورد نیاز در حافظه نهان موجود باشد، Cache Hit یا اصابت رخ می‌دهد و داده با تأخیر کمی در اختیار پردازنده قرار می‌گیرد. در غیر این صورت، Cache Miss یا نقصان رخ می‌دهد و درخواست باید به سطح پایین‌تر حافظه فرستاده شود. هدف اصلی پیش‌واکشی، کاهش تعداد این نقصان‌ها است.

حافظه نهان مورد استفاده در این پروژه از سیاست Write-Back استفاده می‌کند. در این سیاست، داده تغییر یافته ابتدا فقط در حافظه نهان نوشته می‌شود و هنگام جایگزینی بلوک، به سطح پایین‌تر حافظه انتقال

می‌یابد.

۲-۲ پیش‌واکشی حافظه

پیش‌واکشی حافظه یا Memory Prefetching روشی برای انتقال داده‌ها به حافظه نهان پیش از درخواست مستقیم پردازنده است. واحد پیش‌واکشی با مشاهده درخواست‌های فعلی حافظه، تلاش می‌کند آدرس‌های آینده را پیش‌بینی کند.

اگر پیش‌بینی درست باشد، داده موردنیاز پیش از استفاده پردازنده وارد حافظه نهان شده و زمان انتظار کاهش می‌یابد. در نتیجه، نرخ اصابت حافظه نهان و عملکرد برنامه می‌تواند بهبود پیدا کند.

در مقابل، پیش‌بینی اشتباه باعث ورود داده‌های بدون استفاده به حافظه نهان می‌شود. این داده‌ها ممکن است بلوک‌های مفید را از حافظه نهان خارج کنند که به این وضعیت آلودگی حافظه نهان یا Cache Pollution گفته می‌شود. درخواست‌های اضافی همچنین باعث مصرف بیشتر پهنای باند حافظه می‌شوند. بنابراین، دقت پیش‌بینی و تنظیم مناسب پارامترهای پیش‌واکشی اهمیت زیادی دارد.

۲-۳ الگوریتم پیش‌واکشی گام‌دار

الگوریتم پیش‌واکشی گام‌دار یا Stride Prefetching برای تشخیص دسترسی‌هایی استفاده می‌شود که با فاصله‌ای ثابت تکرار می‌شوند. چنین الگویی معمولاً هنگام پیمایش آرایه‌ها و داده‌های ترتیبی دیده می‌شود. در این الگوریتم، اختلاف میان آدرس فعلی و آدرس قبلی محاسبه می‌شود. این اختلاف، گام یا Stride نام دارد. برای مثال، اگر آدرس‌های متوالی برابر با 0x1000، 0x1040 و 0x1080 باشند، گام دسترسی برابر با 0x40 است.

اگر یک گام ثابت چند بار تکرار شود، الگوریتم وجود یک الگوی منظم را تأیید کرده و با اضافه کردن گام به آدرس فعلی، آدرس‌های بعدی را پیش‌بینی می‌کند. تعداد تکرارهای لازم برای تأیید الگو با پارامتر اطمینان و تعداد آدرس‌های تولیدشده با پارامتر درجه پیش‌واکشی مشخص می‌شود.

در پیاده‌سازی این پروژه، آدرس‌ها ابتدا با اندازه خط حافظه نهان هم‌تراز می‌شوند و وضعیت دسترسی‌های هر پردازش به صورت جداگانه نگهداری می‌شود. الگوریتم از گام‌های مثبت و منفی پشتیبانی می‌کند و در صورت تغییر الگو، تولید درخواست‌های پیش‌واکشی را متوقف می‌کند تا الگوی جدید تشخیص داده شود. همچنین، امکان جلوگیری از عبور آدرس‌های پیش‌بینی‌شده از مرز صفحه وجود دارد.

۴-۲ ترجمه آدرس و TLB

برنامه‌ها معمولاً از آدرس مجازی استفاده می‌کنند، درحالی‌که دسترسی واقعی به حافظه اصلی با آدرس فیزیکی انجام می‌شود. واحد مدیریت حافظه یا MMU وظیفه تبدیل آدرس مجازی به آدرس فیزیکی را بر عهده دارد.

برای جلوگیری از مراجعه مکرر به جدول صفحات، ساختاری به نام TLB استفاده می‌شود. TLB یک حافظه کوچک و سریع است که نگاشت‌های اخیر میان آدرس‌های مجازی و فیزیکی را نگهداری می‌کند. اگر نگاشت موردنظر در آن وجود داشته باشد، ترجمه آدرس سریع‌تر انجام می‌شود. آشنایی با این فرایند اهمیت دارد؛ زیرا درخواست‌های حافظه و پیش‌واکشی باید در مرحله مناسبی از مسیر ترجمه و دسترسی به حافظه پردازش شوند.

۵-۲ ساختار MSHR

ساختار MSHR اطلاعات نقصان‌هایی را نگهداری می‌کند که درخواست آن‌ها به سطح پایین‌تر حافظه ارسال شده، اما پاسخ آن‌ها هنوز دریافت نشده است.

اگر در زمان پردازش یک نقصان، درخواست دیگری برای همان بلوک وارد شود، درخواست جدید با درخواست قبلی ادغام می‌شود و درخواست جداگانه‌ای به حافظه فرستاده نمی‌شود. این حالت به‌عنوان اصابت MSHR شناخته می‌شود.

استفاده از MSHR باعث کاهش درخواست‌های تکراری و ترافیک اضافی حافظه می‌شود. نرخ اصابت و نقصان MSHR نیز از معیارهای ارزیابی عملکرد پیش‌واکشی در این پروژه هستند.

۶-۲ معرفی شبیه‌ساز Akita

Akita یک چارچوب شبیه‌سازی رویدادمحور برای مدل‌سازی سامانه‌های رایانه‌ای است. در این شبیه‌ساز، اجزایی مانند حافظه‌های نهان، کنترل‌کننده‌های حافظه و مسیرهای ارتباطی به‌صورت مؤلفه‌های مستقل پیاده‌سازی می‌شوند.

اجزای مختلف از طریق ارسال و دریافت پیام با یکدیگر ارتباط برقرار می‌کنند و تغییرات سامانه با استفاده از رویدادها شبیه‌سازی می‌شوند. ساختار پیمانه‌ای Akita امکان تغییر اجزای موجود یا اضافه کردن

اجزای جدید را فراهم می‌کند.

در این پروژه، واحد پیش‌واکشی به حافظه نهان سطح دوم اضافه شده و تأثیر آن با اجرای برنامه‌های محک و بررسی معیارهای حافظه ارزیابی می‌شود.

۷-۲ ساختار حافظه نهان در Akita

در حافظه نهان Write-Back شبیه‌ساز Akita، درخواست‌های اجزای بالاتر از طریق پورت بالایی وارد می‌شوند. این درخواست‌ها ابتدا توسط Top Parser دریافت شده و بر اساس نوع آن‌ها به مرحله مناسب فرستاده می‌شوند.

مرحله دایرکتوری بررسی می‌کند که بلوک موردنظر در حافظه نهان وجود دارد یا خیر. در صورت اصابه، درخواست به بانک داده فرستاده می‌شود تا عملیات خواندن یا نوشتن انجام گیرد. در صورت نقصان، یک ورودی MSHR ایجاد شده و درخواست از طریق پورت پایینی به سطح بعدی حافظه ارسال می‌شود.

پس از دریافت پاسخ، داده در حافظه نهان قرار می‌گیرد و درخواست‌های منتظر پاسخ داده می‌شوند. اگر برای جایگزینی، یک بلوک تغییر یافته انتخاب شود، داده آن پیش از حذف شدن به سطح پایین‌تر نوشته می‌شود.

در این پروژه، Top Parser درخواست‌های خواندن واقعی برنامه را در اختیار واحد پیش‌واکشی قرار می‌دهد. واحد پیش‌واکشی الگوی آدرس‌ها را بررسی کرده و در صورت تشخیص گام ثابت، آدرس‌های آینده را تولید می‌کند. درخواست‌های ایجاد شده سپس وارد مسیر عادی پردازش حافظه نهان می‌شوند.

فصل ۳

طراحی و پیاده‌سازی واحد پیش‌واکشی گام‌دار

در این فصل، طراحی و پیاده‌سازی واحد پیش‌واکشی گام‌دار در حافظه نهان سطح دوم شبیه‌ساز MGPUSim شرح داده می‌شود. این شبیه‌ساز بر پایه چارچوب Akita و با زبان Go پیاده‌سازی شده است. تغییرات پروژه شامل دو بخش اصلی هستند: نخست، ایجاد زیرساختی برای اتصال یک پیش‌واکشی‌کننده به حافظه نهان و پردازش ایمن درخواست‌های پیش‌واکشی؛ دوم، پیاده‌سازی الگوریتم گام‌دار برای تشخیص دسترسی‌های دارای فاصله ثابت و تولید آدرس‌های آینده.

در طراحی انجام‌شده تلاش شد پیش‌واکشی‌کننده از منطق اصلی حافظه نهان جدا باقی بماند، درخواست‌های واقعی برنامه نسبت به درخواست‌های پیش‌واکشی اولویت داشته باشند و غیرفعال کردن پیش‌واکشی هیچ تغییری در مسیر پایه شبیه‌ساز ایجاد نکند.

۳-۱ بررسی مسیر پردازش درخواست و محل اتصال پیش‌واکشی

حافظه نهان Write-Back مورد استفاده در این پروژه در مسیر `akita/mem/cache/writeback` قرار دارد و درخواست‌های حافظه را در چند مرحله پردازش می‌کند. درخواست خواندن یا نوشتن ابتدا در Top Parser دریافت و به یک تراکنش داخلی تبدیل می‌شود. سپس مرحله دایرکتوری وجود بلوک را در حافظه نهان و MSHR بررسی می‌کند. در صورت اصابت، درخواست برای دسترسی به بانک داده فرستاده می‌شود و در صورت نقصان، واکشی داده از سطح پایین‌تر حافظه آغاز می‌گردد. پس از دریافت پاسخ، مرحله MSHR درخواست‌های منتظر را تکمیل می‌کند.

بررسی این مسیر دو نقطه مناسب برای اتصال پیش‌واکشی را مشخص کرد. نقطه نخست در Top

Parser است؛ زیرا در این مرحله درخواست خواندن واقعی، همراه با آدرس و شناسه پردازش، در دسترس قرار دارد و می‌توان آن را برای آموزش الگوریتم استفاده کرد. نقطه دوم در مرحله دایرکتوری است؛ زیرا در این مرحله می‌توان پیش از ارسال درخواست پیش‌واکشی به سطح پایین‌تر، موجود بودن بلوک در حافظه نهان یا در حال واکشی بودن آن در MSHR را بررسی کرد.

۲-۳ معماری کلی واحد پیش‌واکشی

معماری واحد پیش‌واکشی بر سه اصل استوار است.

نخست، پیش‌واکشی در حالت غیرفعال باید کاملاً از مسیر عادی حافظه نهان جدا باشد. بنابراین، هنگامی که پیش‌واکشی‌کننده غیرفعال است، درخواست‌های ورودی مستقیماً در مسیر اصلی پردازش می‌شوند و هیچ آدرس یا تراکنش پیش‌واکشی تولید نمی‌شود.

دوم، درخواست‌های واقعی برنامه و درخواست‌های پیش‌واکشی باید در تمام مراحل قابل تشخیص باشند. درخواست‌های واقعی با عنوان Demand و درخواست‌های ساخته‌شده توسط پیش‌واکشی‌کننده با عنوان Prefetch در نظر گرفته شدند. برای این منظور، فیلد IsPrefetch به ساختار داخلی تراکنش اضافه شد.

سوم، الگوریتم پیش‌بینی نباید به جزئیات مراحل داخلی حافظه نهان وابسته باشد. الگوریتم تنها آدرس درخواست‌های واقعی را مشاهده می‌کند و فهرستی از آدرس‌های پیشنهادی برمی‌گرداند. تبدیل این آدرس‌ها به تراکنش، اولویت‌بندی آن‌ها و بررسی حافظه نهان و MSHR بر عهده زیرساخت حافظه نهان است.

در حالت فعال، جریان کلی پردازش به این صورت است که ابتدا یک درخواست خواندن واقعی وارد Top Parser می‌شود. پیش‌واکشی‌کننده آدرس آن را بررسی می‌کند و در صورت تشخیص الگوی مناسب، یک یا چند آدرس آینده را تولید می‌کند. این آدرس‌ها وارد صف کم‌اولویت می‌شوند و تنها پس از رسیدگی به درخواست‌های واقعی، وارد مسیر دایرکتوری خواهند شد. اگر بلوک از قبل در حافظه نهان یا MSHR وجود داشته باشد، درخواست پیش‌واکشی حذف می‌شود؛ در غیر این صورت، مسیر عادی نقصان را طی می‌کند.

۳-۳ رابط و پیکربندی پیش‌واکشی‌کننده

برای جدا کردن الگوریتم از زیرساخت حافظه نهان، رابط زیر در فایل `akita/mem/cache/prefetcher.go` تعریف شد:

```
1 type Prefetcher interface {  
2     Inspect(req *memprotocol.ReadReq)  
3     GetPrefetchAddresses() []uint64  
4     Reset()  
5 }
```

تابع `Inspect` هر درخواست خواندن واقعی را در اختیار الگوریتم قرار می‌دهد. تابع `GetPrefetchAddresses` آدرس‌های تولیدشده در آخرین بررسی را برمی‌گرداند و تابع `Reset` تمام وضعیت داخلی پیش‌واکشی‌کننده را پاک می‌کند. برای آزمون مستقل زیرساخت، یک پیاده‌سازی تھی با نام `DummyPrefetcher` نیز ایجاد شد که هیچ آدرسی تولید نمی‌کند. پارامترهای الگوریتم گام‌دار در ساختار زیر نگهداری می‌شوند:

```
1 type StridePrefetcherConfig struct {  
2     Degree          int  
3     ConfidenceThreshold int  
4     HistorySize      int  
5     BlockSize        uint64  
6     PageSize         uint64  
7     PreventPageCrossing bool  
8 }
```

پارامتر `Degree` حداکثر تعداد آدرس‌های تولیدشده پس از تشخیص یک گام را تعیین می‌کند. پارامتر `ConfidenceThreshold` تعداد تکرارهای لازم برای معتبرشناختن گام را مشخص می‌سازد. پارامتر `HistorySize` حداکثر تعداد آدرس‌های اخیر نگهداری‌شده برای هر جریان را محدود می‌کند. همچنین `BlockSize` برای هم‌تراز کردن آدرس‌ها، `PageSize` برای تشخیص مرز صفحه و `PreventPageCrossing` برای فعال یا غیرفعال کردن محدودیت عبور از صفحه استفاده می‌شوند.

در سازنده الگوریتم، مقادیر نامعتبر اصلاح می‌شوند. برای نمونه، درجه و آستانه اطمینان باید دست‌کم برابر یک، اندازه تاریخچه باید دست‌کم برابر دو و اندازه بلوک باید بزرگ‌تر از صفر باشد. این بررسی مانع ایجاد وضعیت‌های نامعتبر در زمان اجرا می‌شود.

۳-۴ پیاده‌سازی الگوریتم پیش‌واکشی گام‌دار

الگوریتم گام‌دار بر این فرض استوار است که اگر اختلاف میان چند آدرس متوالی ثابت باشد، احتمال دارد دسترسی‌های بعدی نیز با همان فاصله ادامه پیدا کنند. پیاده‌سازی انجام‌شده برای هر شناسه پردازش، وضعیت مستقلی نگه می‌دارد تا دسترسی‌های پردازش‌های مختلف با یکدیگر ترکیب نشوند.

۳-۴-۱ وضعیت هر جریان و تاریخچه دسترسی‌ها

وضعیت هر جریان شامل تاریخچه آدرس‌ها، آخرین آدرس معتبر، گام تشخیص داده‌شده و مقدار اطمینان است. ساختار کلی آن به صورت زیر است:

```
1 type strideStreamState struct {
2     history      []uint64
3     lastAddress  uint64
4     hasLast      bool
5     stride       int64
6     confidence   int
7 }
```

این وضعیت‌ها در یک نگاشت بر اساس شناسه پردازش یا PID ذخیره می‌شوند. همچنین، هر نمونه پیش‌واکشی‌کننده فهرستی موقت از آدرس‌های تولیدشده در آخرین فراخوانی نگه می‌دارد.

تاریخچه فقط تا اندازه تعیین‌شده در HistorySize رشد می‌کند. پس از پرشدن آن، قدیمی‌ترین آدرس حذف و آدرس جدید به انتهای تاریخچه اضافه می‌شود. در پیاده‌سازی فعلی، تصمیم اصلی الگوریتم بر اساس آخرین آدرس، گام جاری و مقدار اطمینان گرفته می‌شود و تاریخچه، سابقه محدود آدرس‌های اخیر را نگهداری می‌کند.

۳-۴-۲ هم‌تراز کردن آدرس‌ها

پیش‌واکشی در سطح خطوط حافظه نهان انجام می‌شود، نه در سطح بایت‌های داخل یک خط. به همین دلیل، پیش از محاسبه گام، آدرس درخواست با اندازه خط حافظه نهان هم‌تراز می‌شود. اگر آدرس درخواست را با A و اندازه خط را با B نشان دهیم، آدرس هم‌تراز از رابطه زیر به دست می‌آید:

$$A_{\text{aligned}} = \left\lfloor \frac{A}{B} \right\rfloor B \quad (۱-۳)$$

برای نمونه، با خط حافظه نهان ۶۴ بایتی، تمام آدرس‌های واقع در بازه 0x1000 تا 0x103f به آدرس خط 0x1000 تبدیل می‌شوند. این کار از تشخیص گام‌های کاذب ناشی از دسترسی به بایت‌های مختلف یک خط جلوگیری می‌کند.

۳-۴-۳ محاسبه گام و به‌روزرسانی اطمینان

پس از دریافت نخستین آدرس یک جریان، الگوریتم فقط آن را به‌عنوان آخرین آدرس ذخیره می‌کند و هنوز امکانی برای محاسبه گام وجود ندارد. از درخواست دوم به بعد، گام کاندید از اختلاف آدرس هم‌تراز فعلی و آخرین آدرس معتبر محاسبه می‌شود:

$$S_n = A_n - A_{n-1} \quad (۲-۳)$$

نوع گام به‌صورت علامت‌دار در نظر گرفته شده است تا الگوریتم از حرکت رو به جلو و رو به عقب در حافظه پشتیبانی کند. پیش از انجام تفریق، امکان نمایش اختلاف در نوع علامت‌دار بررسی می‌شود تا سرریز رخ ندهد.

اگر گام جدید با گام ذخیره‌شده برابر باشد، مقدار اطمینان افزایش می‌یابد. اگر گام تغییر کند، الگوریتم گام جدید را جایگزین می‌کند و مقدار اطمینان را برابر یک قرار می‌دهد. تا زمانی که مقدار اطمینان به ConfidenceThreshold نرسیده باشد، هیچ آدرس پیش‌واکشی تولید نمی‌شود. بنابراین، تغییر الگو باعث توقف موقت پیش‌واکشی می‌شود تا گام جدید به تعداد کافی تکرار گردد.

منطق اصلی بررسی هر درخواست را می‌توان به‌صورت خلاصه چنین نمایش داد:

```
1 clear pending addresses
```

```

2 reject nil or zero-size requests
3 align the current address to the cache line
4 update the bounded history
5
6 if this is the first address:
7     save it and stop
8
9 candidate = current - last
10 if candidate is zero:
11     stop without destroying the stable pattern
12
13 if candidate differs from the stored stride:
14     stride = candidate
15     confidence = 1
16 else:
17     increase confidence up to the threshold
18
19 save the current address
20 if confidence reaches the threshold:
21     generate prefetch addresses

```

گام صفر معمولاً زمانی رخ می‌دهد که چند درخواست به بایت‌های مختلف یک خط حافظه نهان تعلق داشته باشند. در این حالت، الگوریتم آدرسی تولید نمی‌کند و الگوی پایدار قبلی را نیز از بین نمی‌برد. همچنین آخرین آدرس معتبر تغییر نمی‌کند تا یک دسترسی تکراری داخل همان خط، محاسبه گام بعدی را مختل نکند.

۳-۴-۴ تولید آدرس‌های پیش‌واکشی

پس از رسیدن اطمینان به آستانه، الگوریتم حداکثر به تعداد Degree آدرس تولید می‌کند. اگر آخرین آدرس واقعی برابر A_n و گام تشخیص داده شده برابر S_n باشد، آدرس پیش‌بینی شده شماره i از رابطه زیر محاسبه می‌شود:

$$\hat{A}_{n+i} = A_n + iS_n, \quad 1 \leq i \leq \text{Degree} \quad (3-3)$$

تولید آدرس‌ها به صورت مرحله‌ای انجام می‌شود؛ یعنی هر آدرس از افزودن یک گام به آدرس قبلی به دست می‌آید. این روش هم برای گام مثبت و هم برای گام منفی کاربرد دارد.

در هر مرحله، معتبر بودن آدرس جدید بررسی می‌شود. اگر افزودن گام مثبت باعث سرریز عدد بدون علامت شود، یا اعمال گام منفی آدرس را به کمتر از صفر برسد، تولید آدرس متوقف می‌شود. همچنین در صورت فعال بودن PreventPageCrossing، تمام آدرس‌های پیشنهادی باید در همان صفحه‌ای قرار داشته باشند که درخواست واقعی جاری در آن قرار دارد. به محض رسیدن به نخستین آدرس خارج از صفحه، تولید ادامه پیدا نمی‌کند.

فهرست آدرس‌های تولیدشده هنگام تحویل به زیرساخت کپی می‌شود. در نتیجه، تغییر احتمالی فهرست دریافت‌شده توسط بخش فراخواننده، وضعیت داخلی الگوریتم را تغییر نمی‌دهد. فراخوانی Reset نیز تمام جریان‌های ثبت‌شده و آدرس‌های موقت را پاک می‌کند.

۳-۵ اتصال الگوریتم به حافظه نهان سطح دوم

نمونه پیش‌واکشی‌کننده از طریق رابط Prefetcher در منابع حافظه نهان نگهداری می‌شود. هنگام ساخت حافظه نهان، اگر پیش‌واکشی فعال باشد، پیکربندی الگوریتم گام‌دار ساخته شده و اندازه خط حافظه نهان نیز در آن قرار می‌گیرد. وابستگی زیرساخت به رابط، امکان استفاده از یک پیش‌واکشی‌کننده ساختگی در آزمون‌ها و جایگزینی الگوریتم را بدون تغییر مراحل اصلی حافظه نهان فراهم می‌کند.

در پیکربندی R9Nano، برای هر بانک حافظه نهان سطح دوم یک نمونه مستقل از پیش‌واکشی‌کننده ساخته می‌شود. بنابراین، وضعیت جریان‌های هر بانک از بانک‌های دیگر جدا است و در داخل هر نمونه نیز وضعیت‌ها بر اساس PID تفکیک می‌شوند.

انتخاب الگوریتم و پارامترهای آن از طریق خط فرمان انجام می‌شود. مهم‌ترین گزینه‌های افزوده‌شده عبارت‌اند از:

- prefetcher - برای انتخاب none یا stride؛

- prefetch-degree - برای تعیین درجه پیش‌واکشی؛

- prefetch-confidence - برای تعیین آستانه اطمینان؛
- prefetch-history-size - برای تعیین اندازه تاریخچه؛
- prefetch-prevent-page-crossing - برای کنترل عبور از مرز صفحه.

مقادیر این گزینه‌ها پیش از اجرای شبیه‌ساز اعتبارسنجی و سپس از بخش Runner به سازنده پیکربندی سخت‌افزار منتقل می‌شوند. در حالت none، پیش‌واکشی‌کننده ساخته نمی‌شود و مسیر پایه حافظه نهان بدون تولید درخواست اضافی اجرا خواهد شد.

۳-۶ ایجاد و پردازش تراکنش‌های پیش‌واکشی

پس از بررسی یک درخواست واقعی، آدرس‌های پیشنهادی الگوریتم در صف داخلی پیش‌واکشی قرار می‌گیرند. ظرفیت این صف با پارامتر PrefetchQueueCapacity کنترل می‌شود و مقدار پیش‌فرض آن در حافظه نهان برابر هشت است. اگر صف به ظرفیت نهایی رسیده باشد، آدرس‌های جدیدی که دیگر فضای کافی برای آن‌ها وجود ندارد وارد صف نمی‌شوند.

در هر چرخه، ابتدا درخواست واقعی پردازش می‌شود و سپس زیرساخت تلاش می‌کند قدیمی‌ترین آدرس پیش‌واکشی موجود در صف را وارد بافر مرحله دایرکتوری کند. اگر بافر در آن چرخه پر باشد، آدرس موجود در صف حذف نمی‌شود و برای چرخه بعد باقی می‌ماند. این ترتیب باعث می‌شود پیش‌واکشی مسیر درخواست‌های واقعی را مسدود نکند.

برای هر آدرس خارج‌شده از صف، یک تراکنش خواندن داخلی ساخته می‌شود که آدرس، شناسه پردازش و اندازه دسترسی را از درخواست واقعی مبنا دریافت می‌کند و فیلد IsPrefetch آن برابر با true قرار می‌گیرد.

مرحله دایرکتوری این تراکنش را در سه حالت بررسی می‌کند:

۱. اگر بلوک در MSHR موجود باشد، واکشی آن قبلاً آغاز شده است. درخواست جدید حذف و شمارنده اصابت پیش‌واکشی در MSHR افزایش داده می‌شود.

۲. اگر بلوک در حافظه نهان موجود باشد، واکشی مجدد لازم نیست. درخواست حذف و شمارنده اصابت پیش‌واکشی در حافظه نهان افزایش داده می‌شود.

۳. اگر بلوک در هیچ‌یک از این دو ساختار موجود نباشد، تراکنش وارد مسیر عادی نقصان می‌شود و شمارنده نقصان پیش‌واکشی افزایش می‌یابد.

یک تراکنش پیش‌واکشی درخواست‌کننده واقعی در پورت بالادست ندارد. به همین دلیل، چه در حالت اصابت و چه پس از تکمیل نقصان، هیچ پاسخ `DataReadyRsp` برای آن به پورت بالادست ارسال نمی‌شود. داده دریافت‌شده فقط در حافظه نهان نصب می‌شود تا در صورت مراجعه بعدی برنامه مورد استفاده قرار گیرد.

۷-۳ ثبت پیش‌واکشی‌های مفید، آلودگی و شمارنده‌های آماری

برای تشخیص مفیدبودن داده‌های پیش‌واکشی‌شده، فیلدی با نام `IsPrefetched` به اطلاعات هر بلوک حافظه نهان اضافه شد. هنگامی که داده حاصل از یک تراکنش پیش‌واکشی در حافظه نهان نصب می‌شود، این فیلد فعال می‌گردد.

اگر یک درخواست واقعی برای نخستین بار به چنین بلوکی اصابت کند، پیش‌واکشی مفید ثبت و فیلد آن پاک می‌شود. پاک‌کردن فیلد باعث می‌شود مراجعه‌های بعدی به همان بلوک دوباره به‌عنوان پیش‌واکشی مفید شمارش نشوند. در مقابل، اگر بلوک پیش‌واکشی‌شده پیش از استفاده توسط درخواست واقعی به‌عنوان قربانی جایگزینی انتخاب شود، یک رویداد آلودگی ثبت می‌شود.

معیار آلودگی در این پیاده‌سازی، تعداد بلوک‌های پیش‌واکشی‌شده‌ای است که بدون استفاده جایگزین شده‌اند. این معیار نشان‌دهنده اشغال غیرمفید فضای حافظه نهان است، اما مستقیماً تعداد نقصان‌های واقعی ایجادشده بر اثر بیرون‌رفتن یک بلوک مفید را اندازه‌گیری نمی‌کند.

برای تفکیک ترافیک واقعی از ترافیک پیش‌واکشی، شمارنده‌های زیر به شبیه‌ساز اضافه شدند:

- تعداد تراکنش‌های پیش‌واکشی واردشده به خط لوله؛
- تعداد اصابت‌های پیش‌واکشی در حافظه نهان؛
- تعداد اصابت‌های پیش‌واکشی در `MSHR`؛
- تعداد نقصان‌های پیش‌واکشی؛
- تعداد پیش‌واکشی‌های مفید؛
- تعداد بلوک‌های پیش‌واکشی‌شده جایگزین‌شده پیش از استفاده؛
- تعداد اصابت‌ها، نقصان‌ها و اصابت‌های `MSHR` برای درخواست‌های واقعی.

این شمارنده‌ها مبنای محاسبه نرخ اصابت و نقصان، پوشش پیش‌واکشی، نرخ استفاده، آلودگی و سایر معیارهای فصل ارزیابی هستند.

۸-۳ آزمون‌های صحت پیاده‌سازی

درستی الگوریتم گام‌دار با آزمون‌های واحد مستقل بررسی شد. این آزمون‌ها حالت‌های زیر را پوشش می‌دهند:

- تشخیص گام مثبت و تولید آدرس بعدی؛
- تولید چند آدرس بر اساس مقدار Degree؛
- پشتیبانی از گام منفی؛
- توقف پیش‌واکشی هنگام تغییر الگو و یادگیری دوباره گام جدید؛
- نادیده گرفتن گام صفر ناشی از چند دسترسی به یک خط حافظه نهان؛
- جلوگیری از عبور از مرز صفحه و امکان غیرفعال کردن این محدودیت؛
- جلوگیری از سرریز و کم‌ریزی آدرس؛
- مستقل بودن وضعیت شناسه‌های پردازش مختلف؛
- محدودماندن اندازه تاریخچه؛
- پاک‌شدن خروجی قبلی پس از درخواست نامعتبر؛
- بازنشانی کامل وضعیت و هم‌تراز شدن صحیح آدرس‌ها.

برای آزمون گام مثبت نیز یک دنباله با فاصله ثابت به الگوریتم داده شد و تولید آدرس بعدی با مقدار مورد انتظار مقایسه گردید. جزئیات این نمونه و خروجی آن در فصل ارزیابی ارائه می‌شود.

علاوه بر آزمون‌های الگوریتم، آزمون‌های یکپارچگی مسیر حافظه نهان نیز انجام شدند. این آزمون‌ها بررسی می‌کنند که درخواست واقعی پس از ورود داده پیش‌واکشی شده به حافظه نهان به آن اصابت کند، درخواست واقعی با پیش‌واکشی در حال انجام برای همان بلوک در یک ورودی MSHR ادغام شود،

درخواست‌های صف در زمان پر بودن موقت بافر از بین نروند و تراکنش‌های پیش‌واکشی پاسخ نامعتبر به پورت بالادست ارسال نکنند.

در مجموع، این آزمون‌ها نشان می‌دهند که الگوریتم قادر به تشخیص گام و تولید آدرس معتبر است و زیرساخت نیز درخواست‌های پیش‌واکشی را با حفظ اولویت درخواست‌های واقعی در مسیر حافظه نهان پردازش می‌کند.

۹-۳ دسترسی به کد منبع

کد کامل پیاده‌سازی واحد پیش‌واکشی گام‌دار، تغییرات اعمال‌شده در شبیه‌ساز، آزمون‌ها و اسکریپت‌های تحلیل نتایج در مخزن گیت‌هاب پروژه در دسترس است:

github.com/atenaraeisi/mgpusim-stride-prefetcher

فصل ۴

ارزیابی و تحلیل نتایج

در این فصل، عملکرد پیش‌واکشی‌کننده گام‌دار^۱ با حالت پایه^۲، یعنی اجرای برنامه بدون پیش‌واکشی^۳، مقایسه می‌شود. هدف اصلی آزمایش‌ها بررسی این موضوع است که پیش‌واکشی تا چه اندازه می‌تواند نقصان‌های حافظه نهان^۴ را کاهش دهد و آیا سربرابر^۵ ایجادشده توسط درخواست‌های اضافی، باعث بهبود یا افت زمان اجرا می‌شود.

۴-۱ روش انجام آزمایش‌ها

برای هر برنامه محک^۶، دو نوع اجرای اصلی انجام شد. در اجرای پایه، پیش‌واکشی‌کننده با مقدار none غیرفعال بود. در اجرای دوم، پیش‌واکشی‌کننده گام‌دار فعال شد و سایر شرایط اجرا بدون تغییر باقی ماند. تمام اجراها در حالت زمان‌بندی‌شده^۷ و با فعال‌بودن گزارش کامل انجام شدند. خروجی شبیه‌ساز ابتدا در فایل SQLite ذخیره و سپس جدول معیارها به قالب CSV تبدیل شد. در مرحله تحلیل، آمار تمام ۱۶ بانک حافظه نهان سطح دوم با یکدیگر جمع شد.

برای جلوگیری از مخلوط‌شدن ترافیک واقعی برنامه با ترافیک پیش‌واکشی^۸، شمارنده‌های درخواست‌های واقعی و پیش‌واکشی به‌صورت جداگانه ثبت شدند. همچنین در تمام نتایج مقدار

Stride Prefetcher^۱
Baseline^۲
Prefetching^۳
Cache Misses^۴
Overhead^۵
Benchmark^۶
Timing Mode^۷
Demand and Prefetch Traffic^۸

prefetch_unaccounted برابر صفر بود؛ بنابراین تمام درخواست‌های پیش‌واکشی در یکی از حالت‌های اصابت در حافظه نهان^۹، اصابت در MSHR^{۱۰} یا نقصان پیش‌واکشی^{۱۱} قرار گرفته‌اند.

۲-۴ برنامه محک و تنظیمات اجرا

دو برنامه محک زیر برای ارزیابی استفاده شدند:

- **ضرب ماتریس:** این برنامه با اندازه‌های $128 \times 128 \times 128$ و $512 \times 512 \times 512$ اجرا شد.

- **FIR:** این برنامه با طول ورودی 65536 و تعداد 16 ضریب اجرا شد.

در مقایسه اصلی، تنظیمات پیش‌واکشی‌کننده به صورت زیر بود:

- درجه پیش‌واکشی^{۱۲} برابر 1

- آستانه اطمینان^{۱۳} برابر 1

- اندازه تاریخچه^{۱۴} برابر 8

- اندازه خط حافظه نهان^{۱۵} برابر 64 بایت

- اندازه صفحه برابر 4 KiB

- جلوگیری از عبور پیش‌واکشی از مرز صفحه^{۱۶}

برای برنامه FIR، درجه‌های 2 و 4 نیز بررسی شدند. در اجرای درجه 4، محدودیت عبور از مرز صفحه غیرفعال شد تا اثر یک پیکربندی تهاجمی‌تر نیز مشخص شود.

در آزمایش مقدماتی، آستانه اطمینان برابر 2 نیز بررسی شد. در این حالت، پیش‌واکشی‌کننده در اجرای ضرب ماتریس فقط 4 درخواست پیش‌واکشی تولید کرد. این نتیجه نشان داد که مقدار 2 برای الگوی

^۹ Cache Hit

^{۱۰} MSHR Hit

^{۱۱} Prefetch Miss

^{۱۲} Prefetch Degree

^{۱۳} Confidence Threshold

^{۱۴} History Size

^{۱۵} Cache Line Size

^{۱۶} Page Boundary

دسترسی این برنامه بیش از حد محافظه کارانه است و فرصت‌های کمی برای پیش‌واکشی ایجاد می‌کند. به همین دلیل، در آزمایش‌های اصلی آستانه اطمینان برابر 1 ثابت نگه داشته شد و اثر تغییر درجه پیش‌واکشی بررسی گردید.

۳-۴ معیارهای ارزیابی

معیارهای اصلی استفاده‌شده عبارت‌اند از:

- زمان اجرای کرنل^{۱۷}؛
- نرخ اصابت و نقصان^{۱۸} درخواست‌های واقعی در حافظه نهان سطح دوم؛
- نرخ اصابت و نقصان درخواست‌های واقعی در MSHR؛
- تعداد کل درخواست‌های پیش‌واکشی؛
- تعداد پیش‌واکشی‌های مفید^{۱۹}؛
- میزان پوشش پیش‌واکشی^{۲۰}؛
- نرخ استفاده از داده‌های پیش‌واکشی‌شده^{۲۱}؛
- تعداد بلوک‌های پیش‌واکشی‌شده‌ای که بدون استفاده جایگزین شدند؛ این مقدار شاخصی از آلودگی حافظه نهان^{۲۲} است.

نرخ اصابت حافظه نهان فقط بر اساس درخواست‌های واقعی برنامه محاسبه شده است:

$$\text{Cache Hit Rate} = \frac{\text{Demand Hits}}{\text{Demand Hits} + \text{Demand Misses}}$$

نرخ استفاده از پیش‌واکشی برابر است با:

^{۱۷} Kernel Execution Time

^{۱۸} Hit and Miss Rates

^{۱۹} Useful Prefetches

^{۲۰} Prefetch Coverage

^{۲۱} Prefetch Usage Rate

^{۲۲} Cache Pollution

$$\text{Prefetch Usage} = \frac{\text{Useful Prefetches}}{\text{Total Prefetch Requests}}$$

پوشش پیش‌واکشی نیز از رابطه زیر محاسبه شد:

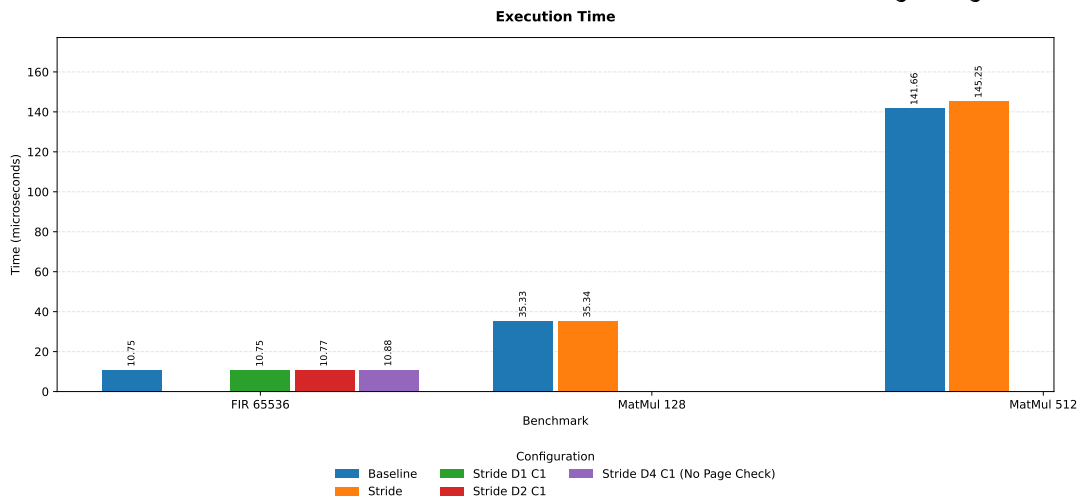
$$\text{Coverage} = \frac{\text{Useful Prefetches}}{\text{Useful Prefetches} + \text{Demand Misses}}$$

۴-۴ مقایسه زمان اجرا

در ضرب ماتریس با اندازه 128، زمان اجرا در حالت پایه برابر 35.326 میکروثانیه و در حالت پیش‌واکشی برابر 35.342 میکروثانیه بود. بنابراین مقدار شتاب^{۲۳} برابر 0.9995 به دست آمد و تفاوت زمان اجرا قابل توجه نبود.

در ورودی بزرگ‌تر ضرب ماتریس، زمان اجرا از 141.658 میکروثانیه در حالت پایه به 145.252 میکروثانیه در حالت پیش‌واکشی افزایش یافت. شتاب این اجرا برابر 0.9753 بود؛ یعنی پیش‌واکشی حدود 2.5% باعث افت کارایی شد.

در برنامه FIR با درجه 1، زمان دو اجرا تقریباً یکسان بود. با افزایش درجه به 2 و 4، زمان اجرا به ترتیب اندکی و حدود 1.2% افزایش یافت. در نتیجه افزایش تعداد درخواست‌های پیش‌واکشی در این برنامه، به بهبود زمان اجرا منجر نشد.



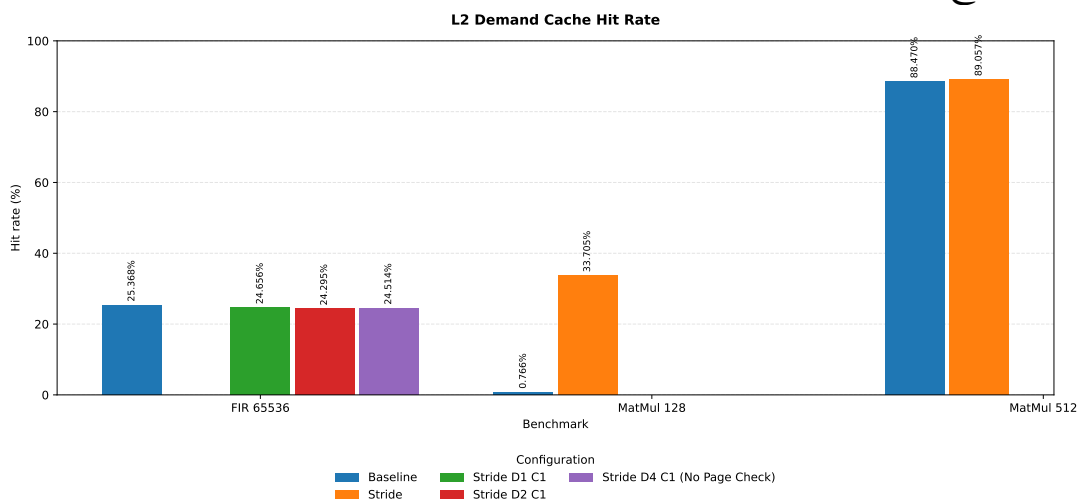
شکل ۴-۱: مقایسه زمان اجرای برنامه‌های محک

۵-۴ مقایسه نرخ اصابت و نقصان حافظه نهان

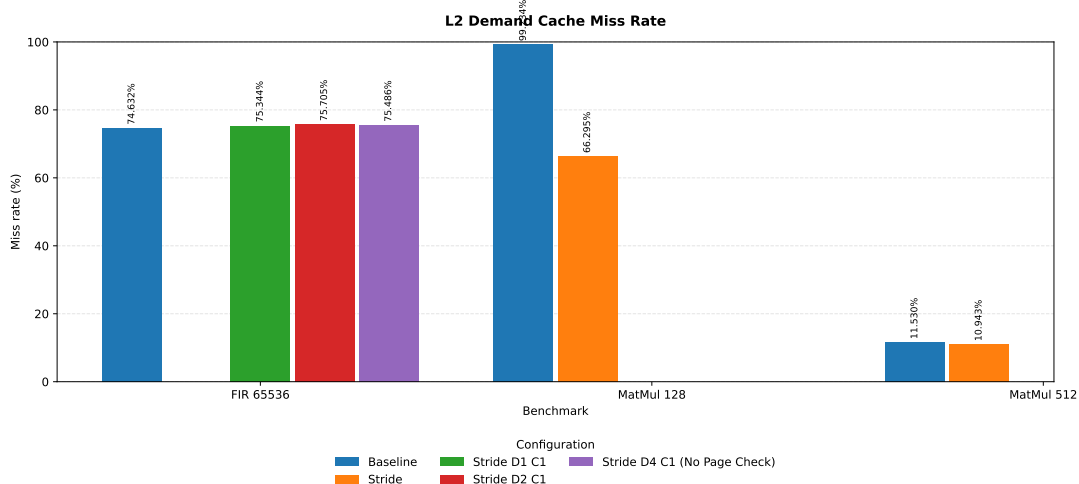
در ضرب ماتریس با اندازه 128، نرخ اصابت Cache از 0.77% به 33.71% افزایش یافت و تعداد نقصانها از 2074 به 1841 کاهش پیدا کرد. با این حال بخش زیادی از افزایش اصابت Cache با کاهش تعداد اصابت‌های MSHR همراه بود و در نهایت زمان اجرا تغییر محسوسی نداشت.

در ضرب ماتریس با اندازه 512، نرخ اصابت از 88.47% به 89.06% رسید. این بهبود کوچک بود و تعداد مطلق نقصانها نیز از 32850 به 33060 افزایش یافت. بنابراین افزایش جزئی نرخ اصابت نتوانست سربار پیش‌واکشی را جبران کند.

در برنامه FIR با درجه 1، نرخ اصابت از 25.37% به 24.66% کاهش یافت. تنظیم‌های دیگر نیز بهبود مشخصی در نرخ اصابت ایجاد نکردند.



شکل ۴-۲: مقایسه نرخ اصابت درخواست‌های واقعی، در حافظه نهان سطح دوم

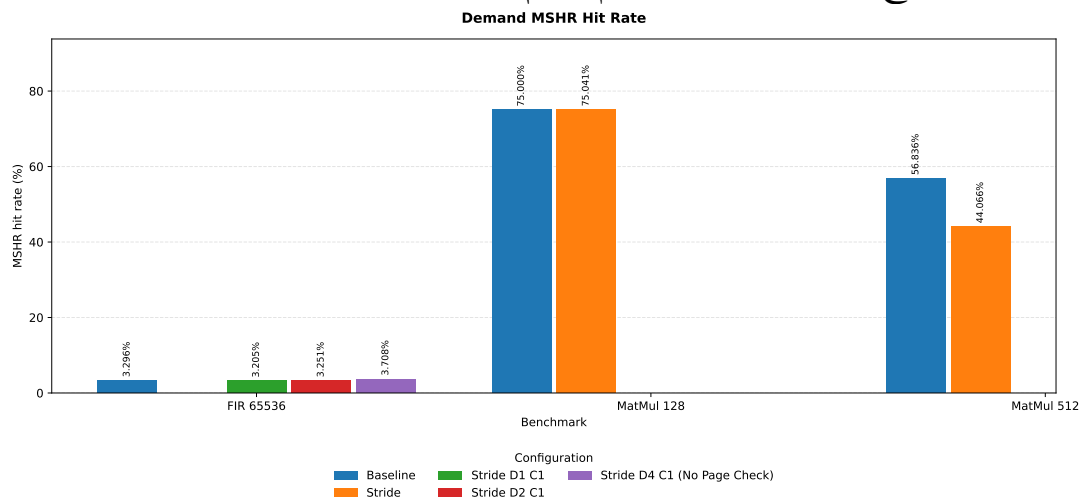


شکل ۴-۳: مقایسه نرخ نقصان درخواست‌های واقعی در حافظه نهان سطح دوم

۶-۴ مقایسه نرخ اصابت و نقصان MSHR

در ضرب ماتریس با اندازه 128، نرخ اصابت MSHR تقریباً ثابت ماند و از 75.00% به 75.04% رسید. در اندازه 512، نرخ اصابت MSHR از 56.84% به 44.07% کاهش یافت و نرخ نقصان آن به 55.93% افزایش پیدا کرد. این تغییر نشان می‌دهد که پیش‌واکشی، ترتیب و هم‌زمانی درخواست‌ها را تغییر داده و تعداد درخواست‌های قابل ادغام^{۲۴} در MSHR کاهش یافته است.

در برنامه FIR نرخ اصابت MSHR در تمام تنظیم‌ها پایین بود و تغییر قابل توجهی ایجاد نشد.



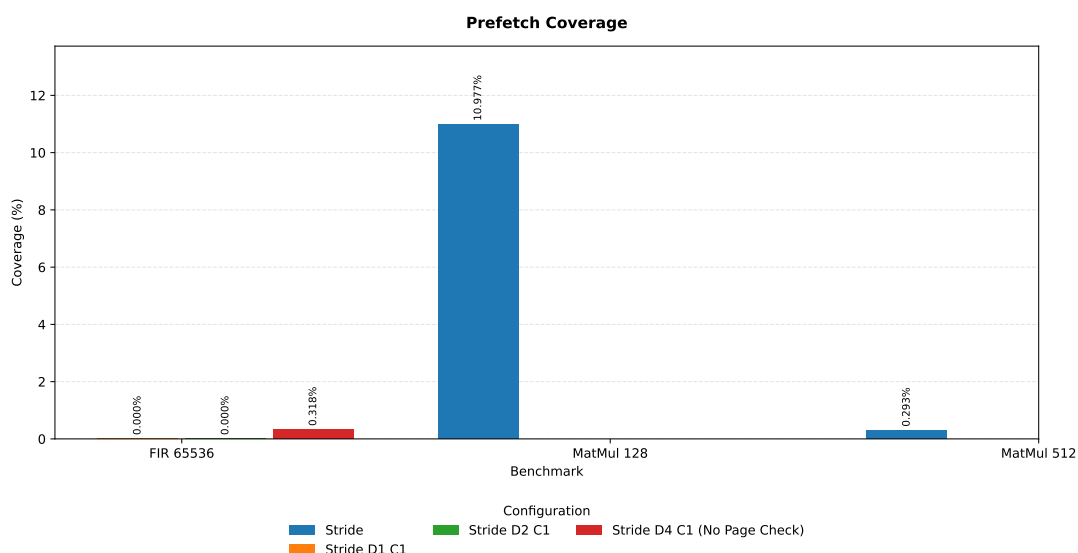
شکل ۴-۴: مقایسه نرخ اصابت درخواست‌های واقعی در MSHR

۷-۴ بررسی پوشش پیش‌واکشی

بیشترین پوشش در ضرب ماتریس با اندازه 128 مشاهده شد و مقدار آن برابر 10.98% بود. در این اجرا، پیش‌واکشی توانست بخشی از نقصان‌های احتمالی برنامه را پوشش دهد.

در ضرب ماتریس با اندازه 512، با وجود تولید تعداد زیادی درخواست، پوشش تنها 0.29% بود. در برنامه FIR نیز پوشش در درجه‌های 1 و 2 صفر بود و در درجه 4 فقط به 0.32% رسید. بنابراین زیاده‌بودن تعداد درخواست‌های پیش‌واکشی الزاماً به افزایش پوشش منجر نشد.

^{۲۴} Mergeable Requests



شکل ۴-۵: مقایسه میزان پوشش پیش‌واکشی

۸-۴ بررسی میزان استفاده از داده‌های پیش‌واکشی شده

در ضرب ماتریس با اندازه 128، از مجموع 2573 درخواست پیش‌واکشی، 227 مورد توسط درخواست‌های واقعی برنامه استفاده شد. نرخ استفاده در این اجرا برابر 8.82% بود.

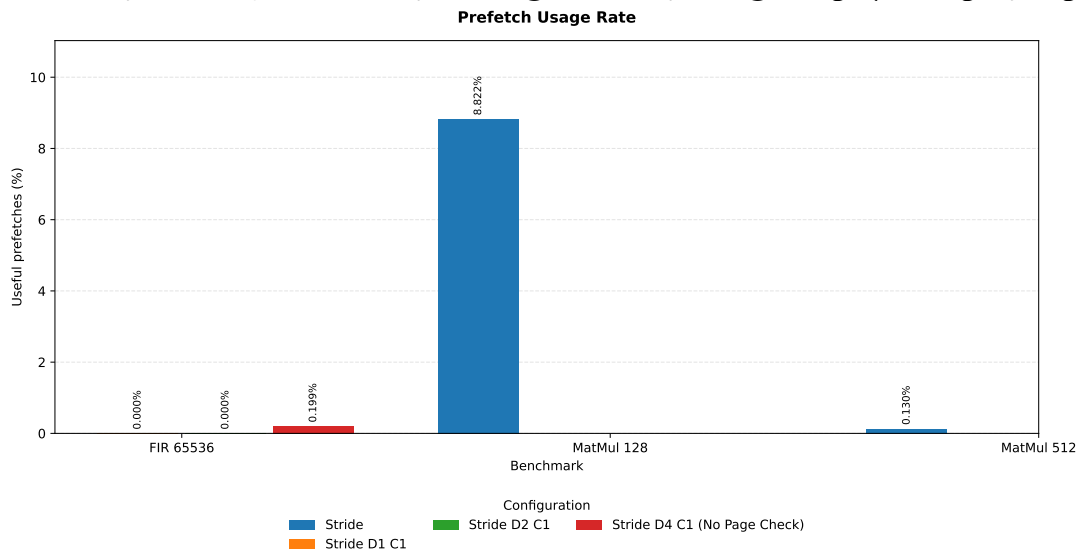
در اندازه 512، تعداد درخواست‌های پیش‌واکشی به 74753 رسید، اما تنها 97 مورد مفید بود. در نتیجه نرخ استفاده فقط 0.13% به دست آمد.

در برنامه FIR، صفر بودن نرخ استفاده در درجه‌های 1 و 2 به این معنا نیست که پیش‌واکشی‌کننده هیچ الگویی تشخیص نداده است؛ زیرا در این دو حالت به ترتیب 847 و 1155 درخواست پیش‌واکشی تولید شد. با این حال، هیچ‌یک از بلوک‌های پیش‌واکشی شده بعداً توسط درخواست‌های واقعی برنامه استفاده نشدند. یکی از دلایل احتمالی این نتیجه این است که چند نوع دسترسی حافظه با هم مخلوط می‌شوند. در پیاده‌سازی فعلی، الگوریتم برای هر شناسه پردازش فقط یک وضعیت نگه می‌دارد. به همین دلیل، آدرس‌های مربوط به داده‌های ورودی، ضرایب فیلتر و بخش‌های مختلف برنامه ممکن است پشت سر هم وارد یک دنباله شوند.

در این حالت، فاصله بین آدرس‌ها همیشه ثابت نمی‌ماند و الگوریتم نمی‌تواند یک گام پایدار تشخیص دهد. در نتیجه، ممکن است آدرسی را پیش‌واکشی کند که برنامه در آن نیاز نداشته باشد.

این موضوع در اجرای درجه 4 نیز دیده شد. با وجود تولید 6540 درخواست پیش‌واکشی، فقط 13 مورد مفید بود و تعداد زیادی از درخواست‌ها برای بلوک‌هایی تولید شدند که از قبل در حافظه نهان قرار داشتند.

بنابراین، افزایش درجه پیش‌واکشی تنها ترافیک اضافی ایجاد کرد و باعث بهبود زمان اجرا نشد.



شکل ۴-۶: مقایسه نرخ استفاده از داده‌های پیش‌واکشی شده

۹-۴ بررسی آلودگی حافظه نهان

در این پروژه، معیار آلودگی برابر با تعداد بلوک‌هایی در نظر گرفته شد که توسط پیش‌واکشی‌کننده وارد Cache شدند، اما پیش از استفاده توسط یک درخواست واقعی، از Cache جایگزین شدند.

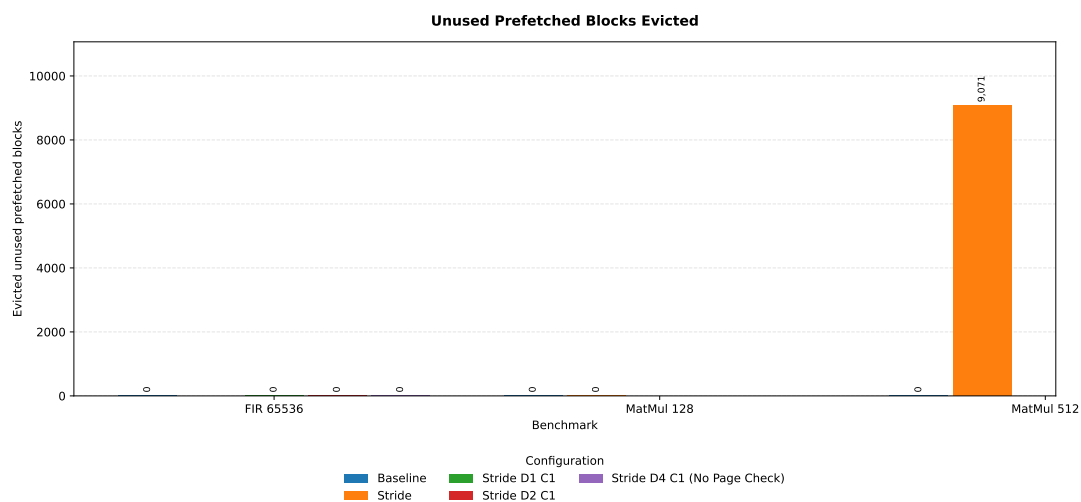
در ضرب ماتریس با اندازه 128 و تمام اجراهای FIR، مقدار این معیار صفر بود. در ضرب ماتریس با اندازه 512، تعداد 9071 بلوک پیش‌واکشی شده بدون استفاده جایگزین شدند. نرخ آلودگی در این اجرا برابر 12.13% بود.

این مقدار نشان می‌دهد که در ورودی بزرگ، بخش قابل توجهی از پیش‌واکشی‌ها فضای Cache را اشغال کرده‌اند، بدون آنکه توسط برنامه مصرف شوند.

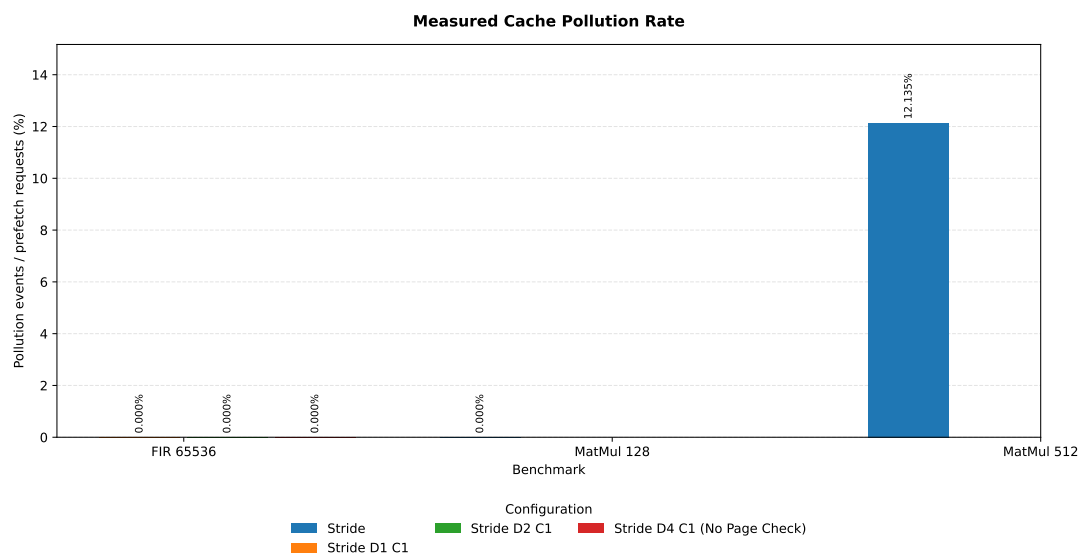
۱۰-۴ تحلیل کلی نتایج

نتایج نشان می‌دهند که پیش‌واکشی‌کننده از نظر عملکرد نرم‌افزاری به‌درستی پیاده‌سازی شده و قادر به تشخیص الگو و تولید درخواست پیش‌واکشی است. با این حال مفیدبودن آن به برنامه محک، اندازه ورودی و پارامترهای انتخاب‌شده وابسته است.

در ضرب ماتریس کوچک، پیش‌واکشی باعث کاهش نقصان‌ها و پوشش حدود 11% شد، اما زمان اجرا



شکل ۴-۷: تعداد بلوک‌های پیش‌واکشی شده جایگزین شده پیش از استفاده



شکل ۴-۸: نرخ آلودگی اندازه‌گیری شده در حافظه نهان

بهبود نیافت. در ضرب ماتریس بزرگ، تعداد بسیار زیادی پیش‌واکشی تولید شد که بیشتر آن‌ها غیرمفید یا تکراری بودند. از 74753 درخواست، تعداد 45370 مورد از قبل در Cache و 10692 مورد از قبل در MSHR موجود بودند. این ترافیک اضافی همراه با آلودگی Cache و کاهش نرخ اصابت MSHR باعث افت کارایی شد.

در برنامه FIR، افزایش درجه پیش‌واکشی باعث افزایش تعداد درخواست‌ها شد، اما تعداد پیش‌واکشی‌های مفید تقریباً صفر باقی ماند. این نتیجه نشان می‌دهد مشکل اصلی کم‌بودن تعداد درخواست‌های پیش‌واکشی نبود، بلکه آدرس‌های پیش‌بینی‌شده با دسترسی‌های آینده برنامه تطابق کافی نداشتند. درهم‌آمیختن چند جریان دسترسی دارای شناسه پردازش یکسان و تغییر مکرر گام مشاهده‌شده می‌تواند یکی از دلایل این رفتار باشد. در نتیجه، پیش‌واکشی تهاجمی‌تر فقط ترافیک حافظه را افزایش داد و به بهبود کارایی منجر نشد.

جدول ۴-۱: خلاصه مهم‌ترین نتایج پیش‌واکشی

برنامه	شتاب	پوشش	نرخ استفاده	نرخ آلودگی
MatMul 128	0.9995	10.98%	8.82%	0%
MatMul 512	0.9753	0.29%	0.13%	12.13%
FIR D1	1.0001	0%	0%	0%
FIR D4	0.9876	0.32%	0.20%	0%

۴-۱۱ اثبات درستی تشخیص گام

برای بررسی مستقل درستی الگوریتم، دنباله‌ای از درخواست‌های خواندن با آدرس‌های 0x1000، 0x1040 و 0x1080 به پیش‌واکشی‌کننده داده شد. از آنجا که اندازه خط حافظه نهان برابر با 64 بایت است، اختلاف میان آدرس‌های متوالی برابر با 0x40 خواهد بود.

پس از رسیدن تعداد تکرارهای این الگو به آستانه اطمینان، پیش‌واکشی‌کننده آدرس 0x10c0 را تولید کرد. این آدرس دقیقاً یک خط حافظه نهان پس از آخرین درخواست واقعی قرار دارد. بنابراین، نتیجه این آزمایش نشان می‌دهد که الگوریتم مقدار گام را به درستی محاسبه کرده و آدرس بعدی را مطابق الگوی تشخیص داده‌شده تولید می‌کند.

جدول ۴-۲: نمونه تشخیص گام و تولید آدرس پیش واکشی

نوع آدرس	مقدار
Demand 1	0x1000
Demand 2	0x1040
Demand 3	0x1080
گام تشخیص داده شده	0x40
آدرس پیش واکشی شده	0x10c0

فصل ۵

جمع‌بندی و پیشنهادها

۵-۱ جمع‌بندی پروژه

در این پروژه، یک رابط قابل تنظیم برای پیش‌واکشی‌کننده به حافظه نهان سطح دوم شبیه‌ساز اضافه شد. سپس الگوریتم پیش‌واکشی گام‌دار پیاده‌سازی شد که با بررسی اختلاف بین آدرس‌های متوالی، الگوی دسترسی ثابت را تشخیص می‌دهد و آدرس‌های آینده را پیش‌بینی می‌کند.

امکان فعال یا غیرفعال کردن پیش‌واکشی‌کننده و تنظیم درجه، آستانه اطمینان، اندازه تاریخچه و کنترل مرز صفحه از طریق پارامترهای خط فرمان فراهم شد. همچنین معیارهای مربوط به درخواست‌های واقعی و پیش‌واکشی از یکدیگر جدا شدند و اسکرپت‌هایی برای استخراج، تحلیل و رسم نمودار نتایج توسعه یافتند.

نتایج نشان دادند که پیاده‌سازی از نظر تولید و پردازش درخواست‌های پیش‌واکشی صحیح است، اما پیکربندی بررسی‌شده در بیشتر اجراها باعث بهبود زمان اجرا نشد. مهم‌ترین دلیل این موضوع، نرخ پایین استفاده از داده‌های پیش‌واکشی‌شده و تولید تعداد زیادی درخواست غیرمفید بود.

۵-۲ محدودیت‌های پروژه

یکی از محدودیت‌های پیاده‌سازی این است که وضعیت الگوریتم برای هر شناسه پردازش نگهداری می‌شود. از آنجا که شمارنده برنامه در رابط پیش‌واکشی موجود نیست، دسترسی‌های مختلف یک پردازش ممکن است در یک جریان مشترک قرار گیرند و تشخیص الگو را دشوار کنند.

همچنین آزمایش‌ها تنها روی دو برنامه محک انجام شدند و نتایج ممکن است برای برنامه‌های دیگر متفاوت باشد. پارامترها نیز فقط در چند حالت محدود بررسی شدند و جست‌وجوی کاملی برای یافتن بهترین ترکیب پارامترها انجام نشد.

معیار آلودگی استفاده‌شده در این پروژه، تعداد بلوک‌های پیش‌واکشی‌شده‌ای را ثبت می‌کند که بدون استفاده جایگزین شده‌اند. این معیار مستقیماً تعداد نقصان‌های واقعی ایجادشده بر اثر خارج‌شدن یک بلوک مفید از Cache را اندازه‌گیری نمی‌کند و تنها شاخصی از اشغال غیرمفید Cache است.

به دلیل محدودیت زمان، جست‌وجوی کامل روی تمام ترکیب‌های درجه پیش‌واکشی و آستانه اطمینان انجام نشد. آستانه اطمینان 2 در یک اجرای مقدماتی بررسی شد، اما به دلیل تولید تعداد بسیار کم درخواست پیش‌واکشی، آزمایش‌های اصلی با مقدار 1 ادامه یافتند.

۳-۵ پیشنهادها برای توسعه آینده

برای توسعه آینده، می‌توان وضعیت پیش‌بینی را به‌جای شناسه پردازش، بر اساس شمارنده برنامه یا ترکیبی از شمارنده برنامه و شناسه پردازش نگهداری کرد. این تغییر باعث جداشدن جریان‌های دسترسی مختلف و افزایش دقت تشخیص گام می‌شود.

همچنین می‌توان درجه و آستانه اطمینان را به‌صورت پویا تنظیم کرد. در این روش، اگر نرخ استفاده پایین یا میزان آلودگی بالا باشد، پیش‌واکشی‌کننده رفتار محافظه‌کارانه‌تری انتخاب می‌کند.

بررسی برنامه‌های محک بیشتر، اجرای چندباره هر آزمایش و مقایسه الگوریتم با روش‌های پیش‌واکشی دیگر نیز می‌تواند ارزیابی کامل‌تری ارائه دهد. در نهایت، افزودن شمارنده‌ای برای اندازه‌گیری مستقیم نقصان‌های ناشی از آلودگی Cache می‌تواند دقت تحلیل نتایج را افزایش دهد.